

Zynq SoC 기반 Plasma Etch OES Virtual Metrology NPU

Wafer-order Baseline + Residual CNN 기반 INT8 NPU

BOSCH Plasma Etch 공정의 OES spectral-temporal 데이터로 Mean Si Etch Depth를 예측하고, Residual CNN inference를 Zynq FPGA의 INT8 NPU로 가속하는 HW/SW Co-design 프로젝트

DATASET

75 WAFERS
8 Lots / LOLO

TARGET WORKLOAD

~1.39M MAC
Residual Tiny CNN

HW TARGET

SHARED 8-MAC
Zynq PL / INT8

최종 예측 구조

Wafer-order Baseline
Linear drift model

+

OES Residual CNN
100 × 128 INT8 map



Predicted Depth
Baseline + Residual

현재 프로젝트 단계

Dataset / Sync / AI input

완료

Baseline / Model compare

완료

Residual CNN / Multi-seed

완료

INT8 PTQ / Fixed Golden

완료

FPGA RTL

이제 시작

AXI / Zynq integration

미구현

Vitis ARM SW

미구현

Real Zybo inference

미구현

이번 회의의 핵심

AI 연구 단계에서 Digital Hardware Implementation 단계로 전환. NPU 구조, arithmetic, memory, RTL 순서, PS/PL interface, 3인 역할을 동결한다.

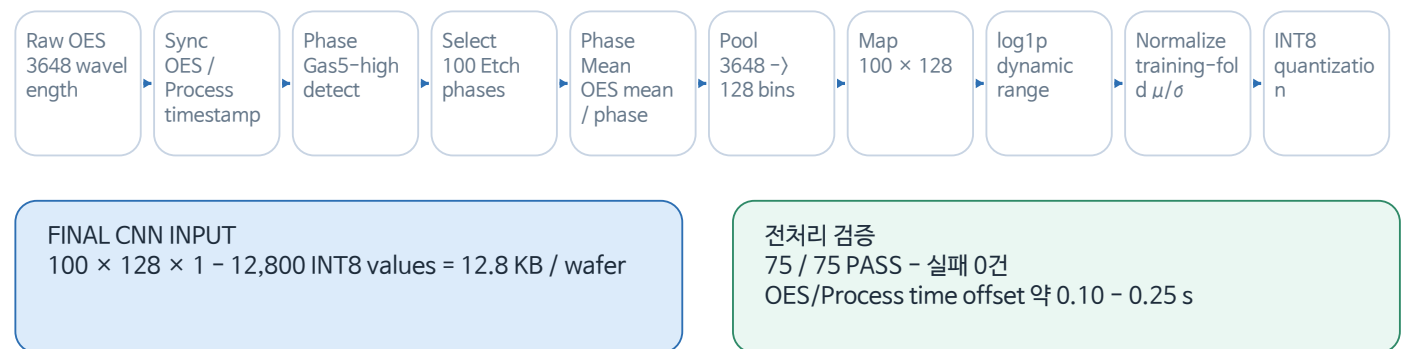
01 / DATA & PREPROCESSING

Dataset과 OES 전처리

75 usable wafers / 8 lots - 100 × 128 INT8 input

WAFER	LOT	OES CHANNEL	PROCESS
75 usable wafers	8 1,2,3,4,5,6,7,9	3,648 185.89 - 883.97 nm	31 variables / 5 Hz
Dataset / Equipment Zenodo: A Multi-Model Dataset for BOSCH Plasma-Etching SPTS Omega i2L DSi Rapier - BOSCH Etch / Passivation cycle		Target 각 wafer의 9-point Si etch depth 평균 Min 39.5544 - Max 40.8170 - Mean 40.2331 - Std 0.3440 μm	

OES 전처리 Pipeline



Gas5 chemistry 표기 주의

Gas5 feature timing이 약 4.5초 Etch 구간과 일치하여 Gas5 $\approx$ SF6 Etch phase로 강하게 추정한다. 단, feature 이름 자체에 chemistry mapping이 명시된 것은 아니므로 “Gas5가 SF6라고 공식적으로 명시되어 있다”고 표현하지 않는다.

02 / MODEL SELECTION

AI 모델을 어떻게 결정했는가

Baseline 비교로 chamber drift를 확인하고 residual learning으로 재정의

LOLO 모델 비교

Model	LOLO MAE	R ²	해석
Mean Baseline	0.2984 μm	-0.0197	정보 없는 기준
Wafer-order Linear	0.1541 μm	0.6998	강한 chamber drift
Direct Tiny CNN	0.2989 μm	-0.0277	absolute prediction 실패
OES Ridge	0.2319 μm	0.2932	OES signal 존재
Wafer + OES Ridge Residual	0.0988 μm	0.8656	정확도 reference
Residual Tiny CNN	0.1163 μm	0.8169 mean	NPU 구현 workload

Direct Tiny CNN: 실패

OES → CNN → Absolute Depth
MAE 0.2989 μm 으로 Mean predictor 수준. Prediction이 평균값 근처로 collapse.

중요한 발견

Wafer 번호만 이용한 Linear Regression이 MAE 0.1541 μm .
Dataset에 wafer processing order에 따른 chamber drift가 강하게 존재.

문제 재정의: Baseline + Residual

Wafer-order Baseline
chamber drift 설명

+

OES Residual
CNN이 잔차만 학습



Etch Depth
최종 예측

모델 선택의 의미

Ridge Residual MAE 0.0988 μm 가 CNN보다 정확하다. CNN 선택 이유는 “가장 정확해서”가 아니다. Ridge는 hand-crafted OES Mean/Slope feature 기반 classical ML accuracy reference이고, CNN은 100×128 spectral-temporal map 자체를 처리하며 FPGA NPU 가속 대상으로 삼은 workload다.

03 / CNN

Final Residual Tiny CNN

5,489 params - ~1,387,536 MAC / inference - INT8 weight ~5.46 KB

Residual CNN 구조

Input
1 × 100 × 128

Conv1 + ReLU
1 → 4, 3 × 3

MaxPool
2 × 2 → 4 × 50 × 64

Conv2 + ReLU
4 → 8, 3 × 3

MaxPool
2 × 2 → 8 × 25 × 32

AvgPool
5 × 4 → 8 × 5 × 8

Flatten
320

FC1 + ReLU
320 → 16

FC2
16 → 1, INT32 residual

PARAMETERS

5,489
trainable

MAC

~1.39M
per inference

WEIGHT

~5.46 KB
INT8

DROPOUT

0.2
training only

Multi-seed Stability
Seeds: 7 / 21 / 42 / 84 / 123
5 seeds × 8 LOLO folds = 40 runs

Mean MAE: 0.1163 μm
Std: 0.0051 μm
Best: 0.1121 μm - Worst: 0.1247 μm
Mean R²: 0.8169

Lot-level note
Lot 5와 Lot 7의 개선 안정성이 상대적으로 약했다.

정확도 해석
Ridge Residual: MAE 0.0988 μm
CNN multi-seed: Mean MAE 0.1163 μm
Accuracy만 보면 Ridge가 우수. CNN은 NPU workload로 채택.

04 / QUANTIZATION

INT8 / Fixed-point Arithmetic Spec

RTL 정답 기준 = Python Fixed-point Golden Model

FP32 CNN MAE

0.1247 μm
LOLO seed 42

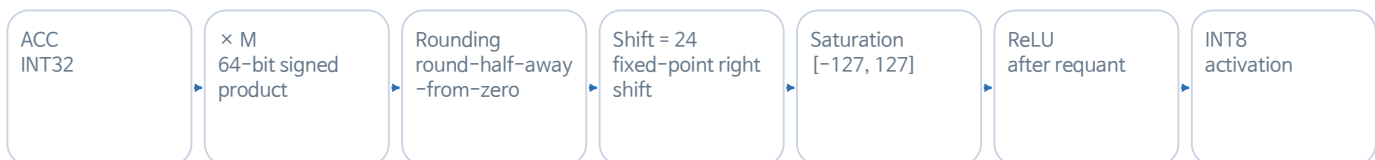
INT8 CNN MAE

0.1232 μm
LOLO seed 42 R^2 0.8056
FP32 0.8020

FPGA 정수 연산 규격



Requantization (Conv1 / Conv2 / FC1)



Layer	M	Shift
Conv1	46,222	24
Conv2	139,797	24
FC1	224,171	24
FC2	requant 없음	-

FC2 / AvgPool 규칙

- FC2는 requant하지 않음
- FC2 INT32 ACC -> PS ARM -> Physical Residual [μm]
- Physical scale: 0.000014877117 μm / count

AvgPool은 $5 \times 4 = 20$ 개 합산 후 /20 rounding 규칙을 Golden과 동일하게 동결.

Fixed-point Golden 검증 및 RTL PASS 기준

기존 float-requant PTQ reference와 integer fixed model을 비교할 때 일부 Conv/Pool intermediate에서 극소수 ± 1 LSB 차이가 있었다. 이 ± 1 LSB는 RTL 허용 오차가 아니다.

RTL 구현은 Python Fixed-point Golden과 bit-exact match를 목표로 한다.

FC1 output mismatch: 0 - FC2 accumulator mismatch: 0 - Final physical output difference: 0.0000000000 μm

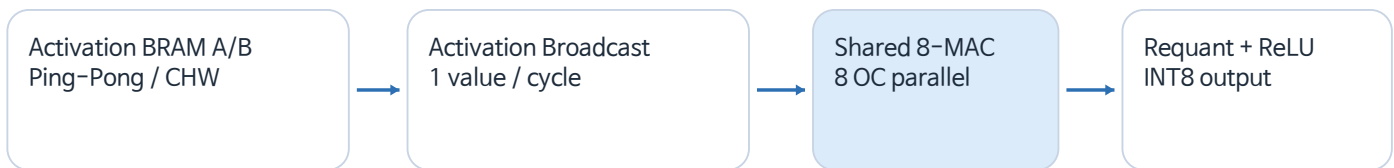
05 / NPU ARCHITECTURE

Shared 8-MAC NPU 아키텍처

Single engine reused by Conv1 → Conv2 → FC1 → FC2

NPU Controller
Layer FSM – Loop Counters – Address Generator – Buffer
Select – Fixed Descriptor

공유 Datapath



8 Output Channel 병렬

Conv2: Cin=4, Cout=8, K=3×3
한 output position의 input term 수 = $4 \times 3 \times 3 = 36$

같은 activation X를 8 lane에 broadcast하고, 각 lane은 서로 다른 output-channel weight를 사용해 ACC0~ACC7을 갱신한다.

36 input term 처리 후 8 output channels가 동시에 완성.

Activation X broadcast – same X to all 8 lanes

W0 → ACC0

W1 → ACC1

W2 → ACC2

W3 → ACC3

W4 → ACC4

W5 → ACC5

W6 → ACC6

W7 → ACC7

Conv + Pool = Non-fused (1차 구현)

Conv1 A→B / Pool1 B→A / Conv2 A→B / Pool2 B→A

Weight lane mapping / layer reuse

동일 (ic, kh, kw)에 대한 8 output-channel weight를 한 64-bit word로 묶는다. Conv2 기준 bits [7:0]=OC0, [15:8]=OC1, ..., [63:56]=OC7.

Conv1은 lane0~3만 enable. FC1은 neuron 0~7 / 8~15의 두 output group으로 처리. FC2는 lane0만 사용.

06 / MEMORY & CONTROL

Memory / Controller 설계

CHW - 32-bit activation packing - 64-bit weight packing - 1-cycle BRAM

Activation CHW element index
 $\text{element_index} = c \times H \times W + r \times W + \text{col}$

실제 BRAM access
 $\text{word_addr} = \text{floor}(\text{element_index} / 4)$
 $\text{byte_sel} = \text{element_index} \bmod 4$

32-bit word byte order

byte0 = bits [7:0]

byte1 = bits [15:8]

byte2 = bits [23:16]

byte3 = bits [31:24]

Packing 예시

elem 0
word 0 / byte 0

elem 1
word 0 / byte 1

elem 2
word 0 / byte 2

elem 3
word 0 / byte 3

elem 4
word 1 / byte 0

Activation BRAM A/B
 A: 16 KiB 이상 권장
 B: 64 KiB 이상 권장
 1-cycle synchronous read
 Cycle N: address / Cycle N+1: data valid

Weight Memory
 $8 \times \text{INT8} = 64\text{-bit word}$
 $\text{Lane0}=\text{W0} \dots \text{Lane7}=\text{W7}$
 ROM order 권장: ic → kh → kw
 kw fastest-changing

Ping-Pong operation table

Operation	Read	Write
PS Input Load	PS → A	A
Conv1	A	B
Pool1	B	A
Conv2	A	B
Pool2	B	A
AvgPool	A	B
FC1	B	A
FC2	A	RESULT

Padding = Zero Injection
 별도 padded tensor를 만들지 않는다.
 Address Generator에서 row/col이
 범위를 벗어나면 activation=0 공급.

Controller = Fixed Descriptor + FSM
 완전 programmable microcode가 아니라
 Generic Engine + Fixed Layer
 Descriptor로 동일 datapath 재사용.

07 / RTL IMPLEMENTATION

RTL 모듈 구성과 실제 구현 순서

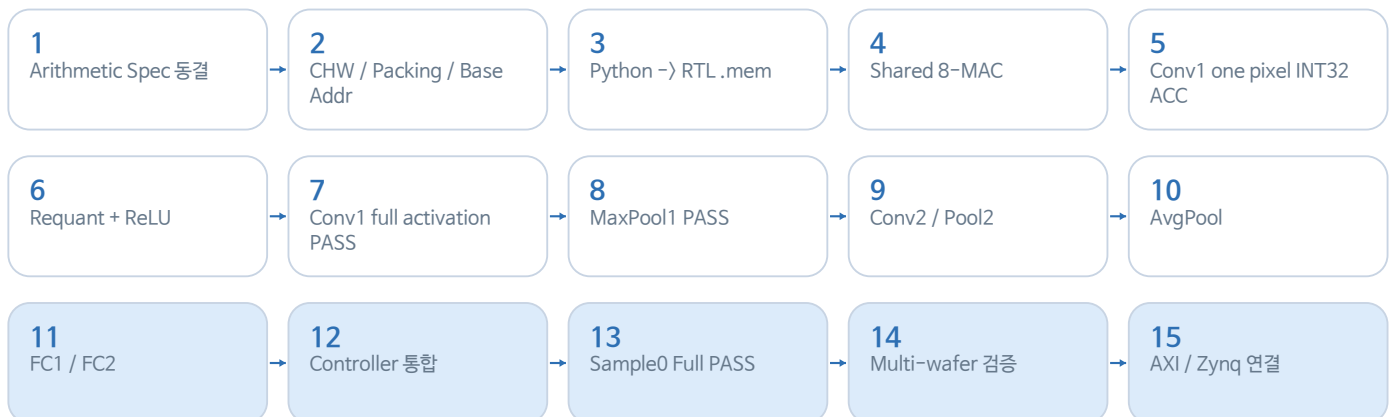
Core bit-exact first -> AXI later

Module	Role
npu_core_top.v	순수 NPU Core Top
npu_controller.v	Layer FSM / Loop Counter / Buffer Select
address_generator.v	Conv / FC / Pool address 생성
mac_array_8.v	Shared 8-MAC
requant_unit.v	64-bit product / rounding / shift / saturation
maxpool_unit.v	2×2 Max Pool
avgpool_unit.v	5×4 sum / 20
activation_bram_A/B	Ping-Pong activation storage
weight_rom / bias_rom	INT8 weight / INT32 bias

개발 원칙: AXI부터 만들지 않는다

먼저 npu_core_top을 clk / reset / start / busy / done / result[31:0] 인터페이스만으로 구현하고 Python Fixed Golden과 bit-exact 완료 후 AXI를 붙인다.

실제 구현 순서



Golden / Verification Single Source of Truth

공식 exporter 예: export_rtl_mem.py

input_sample0.mem, weight.mem, bias.mem, conv1_golden.mem, pool1_golden.mem, conv2_golden.mem, pool2_golden.mem, avgpool_golden.mem, fc1_golden.mem, fc2_golden.txt 생성.

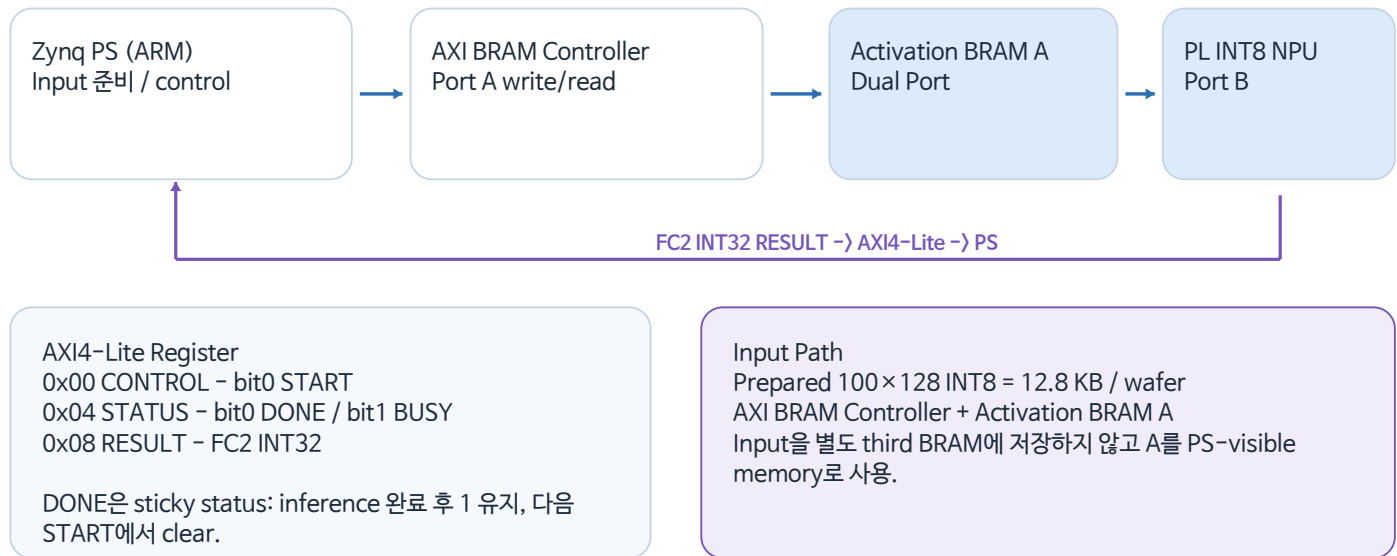
D3 핵심 Gate: Conv1 한 output pixel의 INT32 ACC가 Python Golden과 exact match.

08 / ZYNQ SOC

Vivado -> AXI -> Zynq PS/PL -> Vitis

Input BRAM = Activation BRAM A (PS-visible dual-port)

최종 System Architecture



PS 계산 / Deployment

Baseline

$$\text{baseline} = -0.10232342 \times \text{wafer_order} + 40.77884674$$

Residual

$$\text{residual} = \text{FC2_ACC} \times 0.000014877117 + \text{residual_mean}$$

Final

$$\text{Final Etch Depth} = \text{Baseline} + \text{Residual}$$

wafer_order 정의 확인 필요

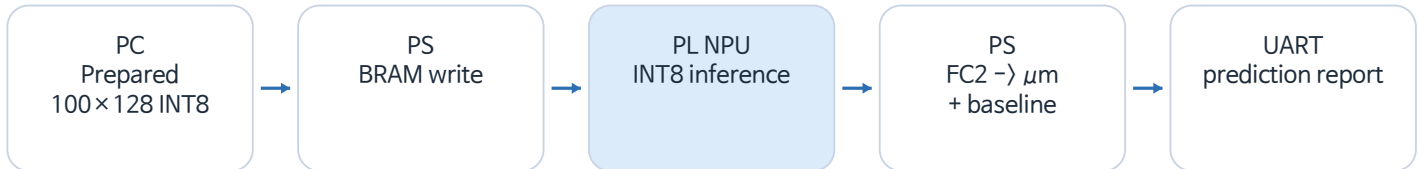
1-based / 0-based / 실제 wafer number / lot 내 processing sequence 중 무엇인지 training/export script에서 확인 후 Spec에 명시. 오정의 시 baseline이 약 0.102 μm 단위로 밀릴 수 있음.

09 / DEMO & BENCHMARK

최종 시연 방법 및 Latency 측정

Prepared INT8 sample 기반 1차 demo - ARM SW vs FPGA NPU 실측

1차 시연 Data Flow



UART / 화면 출력
Lot / Wafer
Baseline
CNN Residual
Predicted Etch Depth
Measured Etch Depth
Absolute Error
NPU Core Latency
End-to-End Accelerator Latency

1차 Demo 범위
원본 NetCDF 전체 전처리를 ARM에서 수행하지 않는다.

PC에서 이미 생성한 100×128 INT8 sample을 사용해 PS -> BRAM -> PL -> RESULT -> PS end-to-end 경로를 먼저 검증한다.

Latency 기준

NPU Core Latency
START -> DONE
순수 PL inference 시간

End-to-End Accelerator Latency
PS BRAM write -> START -> DONE -> RESULT read

Compute floor 해석

모든 8-MAC lane이 매 cycle 100% 활용된 단순 이론 하한: 약 1.73 ms.

현재 선택한 8 output-channel parallel 구조의 순수 MAC schedule:

Conv1 115,200 cycles + Conv2 115,200 + FC1 640 + FC2 16 = 231,056 cycles -> 100 MHz에서 약 2.31 ms.

실제 latency에는 BRAM read latency, address generation, requant, ReLU, pool, write-back, FSM overhead가 추가되므로 2.31 ms보다 길다. Speed-up은 반드시 실제 보드 측정값으로 발표한다.

10 / TEAM EXECUTION

3인 역할 / 통합 / 2주 일정

책임, 선행 관계, 통합 Gate 중심

팀원 A - NPU Datapath
mac_array_8
requant_unit
ReLU
maxpool / avgpool

핵심 목표
정수 연산 규칙을 RTL에서 정확히 구현

팀원 B - Memory / Control
CHW / BRAM Packing
Address Generator
Padding / Ping-Pong
Layer FSM / Golden TB

핵심 목표
Tensor dataflow와 bit-exact 검증

팀원 C - Zynq SoC / SW
AXI4-Lite Wrapper
AXI BRAM Controller
Vivado BD / IP / XSA
Vitis C / UART / latency

핵심 목표
실제 Zybo end-to-end inference

선행 관계와 초기 병렬 착수

B Spec 선행: CHW / packing / weight order / BRAM / loop / padding을 D1에 먼저 동결.
A RTL 선행: D2에 mac_array_8 + requant_unit arithmetic Golden PASS 목표.
C 병렬 착수: NPU Core 완성 전에도 PS + AXI BRAM Controller + BRAM read/write + UART smoke test를 D2부터 진행.

통합 Interface

A ↔ B
MAC input/output - valid - acc clear - weight order

B ↔ C
BRAM map - register map - START/DONE - RESULT format

약 2주 일정

D1 Spec	D2-3 8-MAC + Conv1	D4 Requant / Pool1	D5-6 Conv2 / FC	D7 Full PASS	D8 Multi-wafer / Synth	D9 AXI / Vivado	D10 Vitis / Zybo	Buffer Debug / Bench
------------	--------------------------	--------------------------	--------------------	-----------------	------------------------------	--------------------	---------------------	----------------------------

D2 우선 작업 / 빠른 Gate
A: mac_array_8 + requant_unit → 단일 vector arithmetic Golden PASS
B: BRAM wrapper + Conv1 address generator → address/valid waveform 정확
C: PS + AXI BRAM Controller + BRAM test BD → Vitis write/readback 성공
D3 전후 공통 Gate: Conv1 한 output pixel Golden PASS

11 / D1 SPEC FREEZE

NPU_RTL_SPEC v1.0 동결 항목

오늘 회의의 결과물은 코드보다 먼저 명확한 arithmetic / memory / interface specification

D1 동결 체크리스트

☐

Network - Layer / Tensor shape PDF 구조대로 동결

☐

BRAM read - 1-cycle synchronous

☐

Numeric - Act/Weight signed INT8, Acc/Bias signed INT32

☐

Padding - Address Generator zero injection

☐

Requant product - signed 64-bit intermediate

☐

Parallelism - 8 output channels

☐

Rounding - round-half-away-from-zero

☐

Conv/Pool - 1차 구현 non-fused

☐

Shift / Saturation - 24 / [-127,127]

☐

Controller - Fixed Descriptor + FSM

☐

ReLU - requant 이후

☐

Input memory - Activation BRAM A를 PS-visible dual-port로 사용

☐

AvgPool - /20 rounding Golden 규칙 그대로

☐

Core I/F - start / busy / done / result

☐

Activation layout - CHW

☐

DONE - sticky, next START에서 clear

☐

Packing - $4 \times \text{INT8} = 32\text{-bit}$, Act0=[7:0]

☐

Golden - Python Fixed-point Golden exact match

☐

Weight - $8 \times \text{INT8} = 64\text{-bit}$, Lane0=W0 ... Lane7=W7

☐

Benchmark - Core latency / End-to-End latency 둘 다 측정

발표 / 일반화 한계

- Dataset: 75 wafers / 8 lots로 작음
- LOLO: 동일 BOSCH campaign 내부 unseen-lot feasibility estimate
- Ridge Residual MAE $0.0988 \mu\text{m}$ vs CNN multi-seed $0.1163 \mu\text{m}$
- training MAE $0.0872 \mu\text{m}$ 는 일반화 성능으로 사용하지 않음
- 새로운 Fab / 장비 / Recipe production generalization을 주장하지 않음

프로젝트 성공 기준

1. Python Fixed Golden ↔ RTL Bit-Exact PASS
2. PS → BRAM → PL NPU → RESULT → PS on Zybo
3. 실제 Etch Depth Prediction
4. ARM SW vs FPGA NPU latency 실측
5. LUT / FF / BRAM / DSP resource
6. Timing WNS 확보

wafer_order 정의와 A/B/C 통합 interface를 D1에 확정